PROJECT REPORT

Distributed E-Commerce Microservices Platform

Project Name: Distributed E-Commerce Microservices (Event-Driven Architecture)
Technology: Java 17+, Spring Boot 3, Spring Cloud, Apache Kafka, Apache Avro, PostgreSQL
Project Type: Java Full Stack Cloud Microservices Internship Project
Repository: https://github.com/samyukthasai28/eureka_project

---

1. Abstract

The **Distributed E-Commerce Microservices Platform** is an enterprise-grade, event-driven distributed backend built with **Java 17+**, **Spring Boot 3**, and **Spring Cloud (2023.x)**. The primary purpose of this project is to eliminate monolithic bottlenecks by decomposing the commerce ecosystem into decoupled, independently deployable microservices coordinated via high-throughput asynchronous messaging.

The system utilizes **Apache Kafka** with strictly typed **Apache Avro schemas** to manage event streaming across Order, Inventory, and Payment services with guaranteed partition ordering. Distributed transaction consistency is preserved across isolated PostgreSQL databases using the **Choreography-based Saga Pattern** with automated compensating transactions, backed by the **Transactional Outbox Pattern**.

Dynamic discovery is managed by **Netflix Eureka Server**, externalized settings via **Spring Cloud Config Server**, and edge routing by a non-blocking **Spring Cloud Gateway** enforcing cryptographic **HMAC-SHA256 JWT authentication**. Fault tolerance and distributed observability are achieved via **Resilience4j** circuit breakers, **Micrometer Tracing with Zipkin**, and **Prometheus metrics**. The system is fully containerized using **Docker Compose** and **Kubernetes**.

2. Introduction

In modern cloud applications, high availability, fault isolation, and horizontal scalability are essential requirements. Monolithic architectures suffer from shared database lock contention, tight coupling, and high blast radiuses during deployment.

The Distributed E-Commerce Microservices Platform provides a modern solution by decoupling core business domains into isolated services that communicate asynchronously through events. Each service owns its dedicated PostgreSQL database, strictly adhering to the **Database-per-Service** pattern.

Instead of brittle point-to-point HTTP couplings, business flows like order fulfillment and stock reservation are choreographed over Kafka topics. An API Gateway with JWT verification serves as a single secure entry point, while circuit breakers prevent cascading failures.

3. Objectives

1. To develop a robust, cloud-native distributed microservices architecture using Spring Boot 3.
2. To implement dynamic service discovery and heartbeat monitoring via Netflix Eureka Server.
3. To centralize externalized environment configuration using Spring Cloud Config Server.

4. To enforce reactive, non-blocking routing and cryptographic HMAC-SHA256 JWT validation at the API Gateway.
5. To implement asynchronous event-driven communication using Apache Kafka and Apache Avro schemas.
6. To guarantee partition-level message ordering using aggregate identifiers (Order ID) as Kafka partition keys.
7. To implement Choreography-based Saga distributed transactions with automated compensating rollbacks.
8. To enforce database isolation using PostgreSQL with dedicated databases per service.
9. To build fault tolerance using Resilience4j circuit breakers, retries, rate limiters, and fallback endpoints.
10. To achieve end-to-end distributed observability using Micrometer, Zipkin tracing, and Prometheus metrics.
11. To containerize all services using multi-stage Dockerfiles, Docker Compose, and Kubernetes manifests.
12. To provide an accessible Executive Dashboard UI for real-time topology inspection and transaction tracing.

4. Technologies Used

Technology	Purpose in Platform
Java 17 (OpenJDK 17)	Core programming language utilizing records, pattern matching, and sealed types
Spring Boot 3.3.x	Underlying enterprise backend application framework
Spring Cloud (2023.x)	Distributed microservice coordination (Discovery, Config, Gateway)
Netflix Eureka Server	Centralized dynamic service registry and heartbeat health tracking (:8761)
Spring Cloud Config	Centralized externalized configuration repository (:8888)
Spring Cloud Gateway	Reactive, non-blocking edge reverse proxy and route filter (:8080)
Apache Kafka	Distributed high-throughput, partitioned event streaming message backbone
Apache Avro	Compact, strictly-typed binary schema serialization across services
PostgreSQL	Relational persistence with isolated databases (order_db, inventory_db, payment_db)
Spring Security / JWT	Cryptographic HMAC-SHA256 token verification and stateless API protection
Resilience4j	Fault isolation via circuit breakers, retries, rate limiters, and fallbacks
Micrometer & Zipkin	Distributed tracing, context propagation (B3/W3C), and trace collection (:9411)
Prometheus & Grafana	Time-series metrics scraping and operational telemetry dashboards (:9090 / :3000)
Docker & Docker Compose	Multi-stage container builds and 11-container infrastructure orchestration
Kubernetes (K8s)	Production container deployment, ingress, config maps, and service manifests
Swagger / OpenAPI 3	Interactive REST API documentation and live endpoint testing across all services

5. System Modules

5.1 Service Registry (Eureka Server)

Runs on port **8761** as a dedicated Netflix Eureka Server. Downstream services automatically register their IP, host, and port on startup and send periodic heartbeats. The API Gateway queries Eureka dynamically, routing traffic via lb://service-name without hardcoded URLs.

5.2 Centralized Configuration Server (Config Server)

Runs on port **8888** providing externalized configuration via native filesystem or Git profiles. Microservices fetch their datasource, Kafka, and resilience settings during bootstrap, enabling centralized configuration management without code redeployment.

5.3 API Gateway & JWT Security

Built on Spring Cloud Gateway (port **8080**) utilizing non-blocking Project Reactor. Validates HMAC-SHA256 signatures, expiration claims, and issuer headers on every incoming request. Rejects unauthorized requests with HTTP 401 and routes authorized traffic to microservices through Resilience4j circuit breakers.

5.4 Common Events Module (common-events)

A shared Gradle library containing Apache Avro schemas (OrderPlacedEvent, InventoryReservedEvent, PaymentProcessedEvent, InventoryReleaseEvent, OrderCancelledEvent) and custom binary serializers/deserializers for type-safe Kafka messaging.

5.5 Order Service

Runs on port **8081** handling order placement, state transitions, and PostgreSQL persistence. Implements the Transactional Outbox Pattern and publishes partition-keyed OrderPlacedEvent messages to Kafka. Serves as the primary Saga evaluator for downstream events.

5.6 Inventory Service

Runs on port **8082** managing product inventory and stock allocations. Consumes order events via a dedicated consumer group (inventory-service), performs atomic deductions in inventory_db, emits reservation events, and compensates reserved stock upon receiving InventoryReleaseEvent.

5.7 Payment Service

Runs on port **8083** processing customer transactions. Consumes order events in a dedicated consumer group (payment-service), records payment results in payment_db, and publishes PaymentProcessedEvent or PaymentFailedEvent to Kafka.

5.8 Executive Dashboard UI

Hosted directly on the Gateway (**http://localhost:8080/**), providing an interactive architecture topology map, an instant buy storefront, a live 4-stage Saga Stepper with Kafka payload inspector, and a 4-tab Data Explorer for Orders, Inventory, Payments, and Kafka topics.

6. System Architecture

The system follows a decentralized, event-driven microservices architecture where services communicate asynchronously through Kafka topics rather than synchronous point-to-point HTTP calls.

Distributed Event Choreography Flow:
1. The client sends a JWT-authenticated POST /api/orders request to the API Gateway (:8080).
2. The Gateway validates the JWT token and routes the request to Order Service (:8081) via Eureka load balancing.
3. Order Service records the order as PENDING in order_db and publishes an Avro OrderPlacedEvent to Kafka using the Order ID as the partition key.
4. Inventory Service (:8082) and Payment Service (:8083) consume the event independently in parallel.
5. Inventory Service reserves the requested quantity and emits InventoryReservedEvent.
6. Payment Service captures payment against payment_db and emits PaymentProcessedEvent.
7. Order Service consumes both events; upon dual success, marks the order status as CONFIRMED.
8. If payment fails, Order Service emits an InventoryReleaseEvent to Kafka, triggering Inventory Service to automatically roll back the reserved stock (Automated Saga Compensation).

7. Reliability & Observability Features

• **Choreography-Based Saga Pattern:** Maintains distributed consistency across isolated microservice databases without requiring distributed 2PC locks.

• **Transactional Outbox Pattern:** Guarantees atomicity between local PostgreSQL commits and Kafka message dispatches, preventing phantom events.

• **Partition Key Ordering:** Consistent hashing on Order ID ensures that all state changes for a specific order sequence strictly in order within its Kafka partition.

• **Resilience4j Circuit Breakers:** Gateway routes automatically trip into OPEN state if downstream error rates exceed 50%, serving instant HTTP 503 fallback responses.

• **Exponential Backoff Retries:** Automatically absorbs transient network blips and database reconnection attempts without dropping user requests.

• **Distributed Tracing (Brave & Zipkin):** Injects B3 trace and span headers across HTTP requests and Kafka record metadata, offering end-to-end trace latency visualization.

• **Prometheus & Actuator Telemetry:** Actuator endpoints expose JVM metrics, HTTP request rates, and Kafka producer/consumer lag for Prometheus scraping.

8. Database and Storage

PostgreSQL (order_db): Stores order aggregates, item line entities, outbox event queues, and Saga audit records.
PostgreSQL (inventory_db): Stores product SKU master records, warehouse stock levels, and reservation logs.
PostgreSQL (payment_db): Stores payment transaction records, payment statuses, and customer billing details.
Apache Kafka: Multi-partition persistent message log (order-placed-topic, inventory-events-topic, payment-events-topic, order-compensation-topic) with offset tracking per consumer group.

9. API Documentation

SpringDoc OpenAPI 3 / Swagger UI is embedded in each microservice, providing interactive exploration and live API invocation:
• **API Gateway:** http://localhost:8080/swagger-ui.html
• **Order Service:** http://localhost:8081/swagger-ui.html
• **Inventory Service:** http://localhost:8082/swagger-ui.html
• **Payment Service:** http://localhost:8083/swagger-ui.html
OpenAPI JSON specifications are accessible at /v3/api-docs on each service port.

10. Development Progress & Milestone Breakdown

Development Phase	Delivered Milestones & Scope	Status
Week 1	Eureka Server, Config Server, API Gateway with reactive JWT validation, PostgreSQL schemas, and Payment microservice foundation.	**COMPLETE**
Week 2	Apache Kafka integration, Apache Avro schemas (common-events), Order producer, Inventory/Payment consumers, dedicated consumer groups, partition ordering, and Outbox event store.	**COMPLETE**
Week 3	Choreography-based Saga pattern, automated compensating transactions, Resilience4j circuit breakers, retries, rate limiters, fallback routes, Micrometer/Zipkin tracing, and Prometheus metrics.	**COMPLETE**
Week 4	OpenAPI 3 / Swagger documentation, multi-stage production Dockerfiles, 11-container docker-compose.yml, 13 Kubernetes manifests (k8s/), and GitHub Actions CI/CD workflow.	**COMPLETE**

11. Advantages

- **High Scalability:** Decoupled microservices scale independently based on real-time traffic demands.
- **Fault Isolation:** Failures in inventory or payment services are isolated by circuit breakers without impacting the Gateway.
- **Data Consistency:** Choreographed Saga and Outbox patterns guarantee eventual consistency across distributed databases.
- **Strict Ordering:** Kafka partition keys preserve exact sequencing of business transactions per order.
- **Contract Safety:** Apache Avro schemas enforce binary contract validation and prevent schema drift.
- **Stateless Edge Security:** Cryptographic HMAC-SHA256 JWTs eliminate server-side session memory at the gateway.
- **Operational Visibility:** Zipkin and Prometheus provide deep insight into distributed trace latencies and bottlenecks.

12. Future Scope

- Integration with HashiCorp Vault for enterprise secret lifecycle management and automated key rotation.
- Implementation of CQRS (Command Query Responsibility Segregation) with read-optimized Elasticsearch indices.
- Debezium Change Data Capture (CDC) streaming directly from PostgreSQL write-ahead logs to Kafka.
- Real-time WebSocket / Server-Sent Events (SSE) notification streaming to client frontend applications.
- Canary and Blue-Green deployment automation via ArgoCD within a production Kubernetes cluster.
- Multi-region active-active Kafka cluster replication (MirrorMaker 2) for disaster recovery.

13. Conclusion

The **Distributed E-Commerce Microservices Platform** provides an enterprise-ready reference implementation of modern cloud-native architecture. By combining **Java 17**, **Spring Boot 3**, **Spring Cloud**, **Apache Kafka**, **Apache Avro**, **PostgreSQL**, **Resilience4j**, **Micrometer/Zipkin**, **Docker**, and **Kubernetes**, the project effectively addresses key distributed computing challenges—including eventual consistency without distributed locks, fault isolation, asynchronous event streaming, and full-stack observability.

The project equips engineering teams with practical expertise in microservice decomposition, event-driven architectures, choreographed transactions, and enterprise container deployment.

Project: Distributed E-Commerce Microservices Platform
Repository: https://github.com/samyukthasai28/eureka_project
Development Stack: Java 17+, Spring Boot 3, Spring Cloud, Apache Kafka, PostgreSQL